Webfejlesztés II.

Webprogramozás, JavaScript

Béres Gábor, Fehér Róbert, Kis Balázs

Verzió 1.0

Webprogramozás története I.

- A HTML 1993-as végleges megjelenését követően, a weben lehetőség nyílt dokumentumok egyszerű publikálására, illetve azok feldolgozására.
- Habár eredetileg az egyszerű dokumentum megosztás volt a cél, a 90-es évek közepére sokan felismerték a weboldalakban rejlő további lehetőségeket.
  - Ez lett az interakcióra való törekvés, miszerint a sima dokumentum helyett, interaktív oldalak jöjjenek létre.

Webprogramozás története II.

- Maga a HTML ugyan nem volt képes interakcióra, de megjelentek olyan kiegészítő technológiák (CSS, JavaScript, szerveroldali nyelvek) melyek használatával a megjelenítést, a felhasználóval való kapcsolattartást, továbbra is HTML segítségével lehet megoldani.
- Már csak dinamikus feldolgozókra volt szükség, amik a „statikus” HTML-en keresztül „dinamikus” adatot közölnek, illetve fogadnak.
  - Megjelentek a webprogramozási nyelvek.

Webprogramozás története III.

- A webprogramozási nyelvek segítségével dinamizálható a weblap, melyen így a felhasználóknak jóval több lehetősége nyílik különböző interakciókra, mint egy statikus weboldalon.
- Magukat a webprogramozási nyelveket is meg lehet különböztetni működésük és működési területük alapján.

Webprogramozás története III.

Szerveroldali programozás

- Működésük alapja, hogy a program, alapvetően a szerveren fut le, mely a felhasználótól érkező adatokat feldolgozza, az oldalt pedig dinamikus adatokkal tölti fel.
- Ismertebb technológiák, nyelvek:
  - ASP.NET, PHP, Java, Perl, stb.

Kliensoldali programozás

- Működésük alapja, hogy a program alapvetően a kliens számítógépén fut le, mely a weboldal interaktív megjelenéséért felelős, mint pl.: automatikus adatfrissítés, animációk, stb.
- Ismertebb technológiák, nyelvek:
  - JavaScript, ~~Flash (ActionScript), Silverlight~~, Java FX, stb.

Kliensoldali programozás

Tulajdonságai, előnyei

- Eseményvezérlés
  - A kliensoldali nyelvek eseményvezéreltek.
- Valós idejű lefutás
  - Mivel a program a felhasználó gépén fut, így valós időben tud futtatni programrészeket.
- Grafikai felkészítés
  - A legtöbb ilyen nyelv, a minél látványosabb lefutás érdekében sok grafikai optimalizációt tartalmaz.

Nehézségei, hátrányai

- Biztonsági kockázatok
  - Az adat és kódbiztonság nehézkes megoldása.
  - Nehezen védhető programkód.
- Kliens függőség
  - A kliensen futó program függése a kliens számítógépétől.
- Nehézkes optimalizáció
  - A különböző kliensek, különböző lefutására való odafigyelés.

Szerveroldali programozás

Tulajdonságai, előnyei

- Szerveren való futtatás
  - A felhasználó nem látja magát a programot, csak annak végeredményét.
- Lefutás csak újratöltéskor
  - Klasszikus szerver oldali feldolgozás esetén a program futása nem folytonos, csak akkor fut le, amikor az oldal (újra) betöltődik.
- Adatkiszolgálásra optimalizálva
  - Sok, a weben történő adatkiszolgálást segítő technológiát ötvöznek.

Nehézségei, hátrányai

- Eseményvezérlés hiánya
  - A felhasználó ugyan dinamikus adatokat lát, de az oldal újratöltésig alapvetően statikus.
- Lefutás biztonsága
  - Mivel a program a szerveren fut, így a program törhetősége egyben a szerver törhetősége is.

Webprogramozás története IV.

- Összességében elmondható, hogy a két webprogramozási irányzat, különböző célok elérésében használatos, és a kettő egymástól nem elválasztható felhasználás tekintetében.
- Minden modern weboldal, a két technológiát vegyítve használja, így a kliens- és szerveroldali nyelvek az arra megfelelő alkalmazási területre használva, szinkronban vannak beépítve a weboldalba.

JavaScript

A JavaScript programozási nyelv története, alapvető működése, használata

JavaScript I.

- A JavaScript egy alapvetően kliens oldalon, a böngészőben futtatható programozási nyelv.
- A Netscape egyik munkatársa fejlesztette ki 1995-ben, eredeti neve Mocha, majd LiveScript volt.
- Általános tévedés, hogy a JavaScript a Java programozási nyelv egy származtatott, átfejlesztett verziója, ami azonban nem igaz. A JavaScript név a Java programozási nyelvhez szintaktikai szinten való hasonlóságra utal, ami azonban ebben ki is merül.

JavaScript II.

- A szabványosított változatát ECMA Script-nek hívják, amiből a 3. verzió lett széleskörben elterjedt (1999).
- A programozási nyelv tulajdonságai:
  - Gyengén típusos
    - Az egyes adatok típusait nem a programozó, hanem a fordító dönti el.
  - Nincs szükség előfordításra
    - Habár keretrendszer elvű a nyelv, ellentétben a .NET, JVM keretrendszerekkel, a programot közvetlenül lehet futtatni, így nincs szükség köztesnyelv előállítására.

JavaScript III.

- Objektum-Orientált (Prototípus alapú)
  - Prototípus-Orientált programozási nyelv, ami működésében hasonlít az OO felépítéshez az öröklődés tekintetében. Azonban ebben az esetben nem osztályok hierarchiájáról beszélünk, hanem objektumok hierarchiájáról, azaz objektum származik objektumtól, nem osztály az osztálytól.
- Létezik szerveroldali változata is
  - Ugyan nem pontosan a JavaScript, de készült szerveroldalon felhasználható változata is.
  - Ilyen például a Node.js

JavaScript IV.

- A JavaScript programot alapvetően HTML kódba ágyazva használjuk.
- Elhelyezkedése 3 helyen lehet:
  - HTML Head tag-ben <script>…</script> tag-ek között.
  - HTML egyik tag-jének hozzárendelt eseményében (ide általában függvény, metódus hívásokat szokás elhelyezni).
  - Külön fájlban:
    - *.js fájlban elhelyezve.

JavaScript V.

- Külön fájl esetén, a CSS-el ellentétben, nem a link, hanem a script tag segítségével kell csatolni a HTML oldalhoz:
  - <script ~~type=”JavaScript”~~ src=”xyz.js”></script>
- A külön fájlban metódusokat, funkciókat, esetleg újabb prototípusokat lehet létrehozni, melyeket a HTML-ben fel lehet később használni, bizonyos események bekövetkezése esetén.

JavaScript nyelvi alapjai

A JavaScript programozási nyelv elemei, használata

JavaScript nyelvi alapjai I.

- Mivel a JavaScript a Java nyelvhez hasonlít kinézetét tekintve, így csakúgy mint a Java és a C#, a JavaScript is C alapú nyelv.
  - Ami azt eredményezi, hogy a C nyelvben megszokott és onnan örökölt felépítésű vezérlési szerkezetekkel, operátorokkal találkozhatunk.
- Mivel a nyelv gyengén típusos, a változók típusát a fordító jelöli ki. A változók deklarálása a következőképpen néz ki:
  - var <változónév>;
  - let <változónév>;

JavaScript nyelvi alapjai II.

- Alapvető típusok JavaScript-ben:
  - String
  - Number
  - Boolean
  - Null → „szándékosan nincs értéke”
  - Undefined → „nincs érték hozzárendelve”
  - Symbol
  - Object
  - Array

JavaScript nyelvi alapjai III.

- Szöveggé konvertálás:
  - 2 féleképpen történhet:
    - String(változó / érték);
    - változó.toString();
  - Mindkettő visszaadja szövegbe konvertálva az adott értéket.
- Számmá konvertálás
  - Number(változó / érték);
  - parseInt(változó / érték);
  - parseFloat(változó / érték);
    - Mindhárom számmá konvertál.

JavaScript nyelvi alapjai IV.

- Operátorok:
  - Az operátorok nagy része C nyelvekben is fellelhető, azonban a gyengén típusosság miatt bevezetésre került egy új operátor is:
    - === és a !==
      - Feladata, hogy csak úgy mint az == és a != összehasonlítsa a két értéket, azonban ez megköveteli, hogy a típusuk is azonos legyen.

JavaScript nyelvi alapjai V.

- A document objektum
  - A document objektumon keresztül elérhetővé válik a teljes HTML kód ami az oldal szerkezetét adja.
  - Segítségével a JavaScript hozzáfér a szerkezethez, és abból adatot tud kinyerni, illetve hozzáadni.
  - Fontosabb függvényei:
    - getElementById(”Id”);
      - Visszaadja a megadott Id-val ellátott elemet.
    - getElementsByTagName(”Tag”);
      - Visszaadja a megadott nevű összes tag-et.
    - getElementsByClassName(”Class”);
      - Visszaadja a megadott Class-el ellátott elemeket.
    - write(”Szöveg”);
      - Az adott helyen a DOM fába illeszti a megadott szöveget, mely így megjelenik.

JavaScript nyelvi alapjai VI.

- createElement(”Tag”);
  - Segítségével létrehozható egy megadott nevű tag.
  - Pl.:
    - var tag = document.createElement(”p”);
- createTextNode(”Szöveg”);
  - Létrehoz egy element szintű szöveget.
  - Pl.:
    - var szoveg = document.createTextNode(”Helló Világ!”);
- További függvénylehetőségek:
  - http://www.w3schools.com/jsref/dom_obj_document.asp

JavaScript nyelvi alapjai VII.

- A visszakapott, kinyert szerkezeti elemek (Tag-ek) tulajdonságai:
  - A JavaScript eléri az összes a HTML-ben használt attribútumot, illetve tartalmat a következőképpen:
    - *element*.InnerHTML;
      - Az aktuális elem belső szerkezeti részét (a kezdő és záró tag közti részt) lehet vele kiolvasni, vagy módosítani.
    - *element*.OuterHTML;
      - A teljes tag-et visszaadja, a belső szerkezettel együtt.
    - *element*.attributes;
      - A megadott elem összes tulajdonságát lehet vele lekérdezni.
    - *element*.setAttribute(”attribute”,”value”);
      - A megadott elem, megadott attribútumának lehet vele értéket adni.
    - *element*.getAttribute(”attribute”);
      - A megadott elem, megadott attribútum értékének kiolvasása.

JavaScript nyelvi alapjai VIII.

- *element*.appendChild(tag);
  - Belefűz egy létrehozott tag-et egy már létező, meglévő tag-be.
  - Pl.:
    - let pelem = document.createElement(”p”);
      let pszoveg = document.createTextNode(”Szöveg”);
      pelem.appendChild(pszoveg);
- *element*.removeChild(tag);
  - Kitöröl egy megadott elemet a megadott tag alól.
- *element*.replaceChild(mire, mit);
  - Kicseréli az adott elemet, egy másikra, a megadott elemen belül.
- Természetesen az egyes elemeknek is további beállításai lehetségesek.
- További függvények:
  http://www.w3schools.com/jsref/dom_obj_all.asp

JavaScript nyelvi alapjai IX.

- Metódusok, funkciók:
  - Mivel a JavaScript gyengén típusos nyelv, így a funkciók, metódusok létrehozása és működése eltér az eddig megszokottaktól:
    - Akár metódust, akár funkciót szeretnénk létrehozni a function kulcsszó segítségével tudjuk megoldani:
      - function <metódus/funkció neve>(<paraméterek>)
        {
        …
        }
    - A funkció és metódus közti különbséget nem kell kód szinten jelölni, csak az dönti el, hogy van-e visszatérés az adott ágon, vagy nincs.
      - Lehetséges olyan funkció írása, aminek bizonyos esetben van visszatérése, és bizonyos esetben nincs, illetve a visszatérés típusdefiníciója nélkül bármilyen típussal visszatérhet egy-egy függvény.

JavaScript nyelvi alapjai X.

- Események:
  - Mivel a JavaScript eseményvezérelt, így a felhasználói műveleteket események segítségével tudjuk elérni, azokra azokon keresztül tudunk reagálni.
  - Új események definiálására is van lehetőség.
  - Az eseményekre való feliratkozás a HTML-ben történik a megadott komponens, megadott eseményén.

JavaScript nyelvi alapjai XI.

- Általános HTML-ben kiváltható események:

onchange	Az elem megváltozott.
onclick	A felhasználó az elemre kattintott.
onmouseover	A felhasználó az elem felé navigált (egérrel).
onmouseout	A felhasználó elnavigált az elemről (egérrel).
onkeydown	A felhasználó leütött egy billentyűt.
onload	A böngésző befejezte az oldal betöltését.

- A feliratkozás általános formája:
  - <tag on…=”js kód”…>

Feladat VII.

- Bővítsük a tanulói névsor megoldásunkat (előző diasor VI. feladat) úgy, hogy az adatbeviteli gombra kattintás hatására az új tanuló kerüljön be a táblázatba.

Feladat VIII.

- Készítsünk weboldalt, ami egy alapvető számológépet valósít meg.
- A felhasználó két beviteli mezőbe tudja felvinni a két számot, és egy legördülő menüből tudja kiválasztani a műveletet, és egy gomb segítségével tudja elindítani a számolást.
- Egy JavaScript program végezze el a számítást, és írja vissza az eredményt az oldalon egy felsorolásba, ahol az összes eddigi művelet és eredmény is látszik.
- Az oldalnak adjunk egy alap design-t is.

JavaScript nyelvi alapjai XII.

- A window objektum
  - Segítségével a különböző ablakok megjelenését tudjuk szabályozni.
  - Fontosabb függvényei:
    - open(”url”,[”név”], [”paraméterek”]);
      - Új lapot nyit, a megadott url-el.
      - Az ablaknak név is adható, melyen keresztül elérhetővé válik, így további komponenseket rakhatunk bele, vagy módosíthatunk benne.
    - close();
      - Bezárja az adott lapot.
      - Csak window objektumnál működik.

JavaScript nyelvi alapjai XIII.

- window.location.href;
  - Segítségével lekérdezhető illetve beállítható, hogy mi legyen az adott ablak url-je, hova ugorjon a böngésző az adott lapon.
- window.location.assign(”url”);
  - Beállítható, hogy milyen oldalra ugorjon a böngésző.
- window.history.back();
  - Visszalépés (Vissza gomb)
- window.history.forward();
  - Előre lépés (Előre gomb)
- window.location.replace(”url”);
  - Beállítható, hogy milyen oldalra ugorjon a böngésző, de egyúttal felülírja historyt.

JavaScript nyelvi alapjai XIV.

- A window object időzítési lehetőségei:
  - window.setInterval(”funkció”, milliszekundumok);
    - Mint egy időzítő, az adott funkció x milliszekundumonként le lesz futtatva.
  - window.clearInterval(id);
    - Leállítja az időzítőt, az azonosító alapján.
  - window.setTimeout(”funkció”, milliszekundumok);
    - Egyszeri időzítés, az adott funkció, egyszer lesz lefuttatva x milliszekundum után.

JavaScript nyelvi alapjai XV.

- A window object dialógusai:
  - window.alert(”Szöveg”);
    - Egy szimpla figyelemfelhívás dialógus ablak, egy oké gombbal ellátva.
  - window.confirm(”Kérdés”);
    - Egy döntési ablak, Ok és Mégsem gombokkal ellátva.
  - window.prompt(”Kérdés”, [”Alapértelmezett válasz”]);
    - Egy InputBox, adatot lehet vele bekérni.

Feladat IX.

- Bővítsük ki előző programunkat, ami az oldal betöltése után 10 másodperccel megkérdezi a felhasználót, hogy meg kívánja-e nézni az aktuális tanfolyami listát.
  - Amennyiben a felhasználó meg szeretné nézni, átirányítja az oktatóközpont oldalára.
  - Amennyiben nem szeretné megnézni, a H1-es tag színét pirosra állítja.

Önálló otthoni feladat I.

- Készítsen weboldalt, melyen egy tennivalólistát tudunk kezelni.
- Az oldal megnyitásakor bekérünk a felhasználótól egy nevet, majd lehetősége van felvinni tennivalókat egy táblázatba.
- Egy tennivaló bejegyzésről 3 dolgot adhat meg:
  - Megnevezés
  - Dátum
  - Fontosság {nem fontos, normál, fontos}
- A táblázatban jelenjen meg a megnevezés és a dátum, és a fontosságnak megfelelő színnel legyen kiírva:
  - Nem fontos – kék, normál – fekete, fontos - piros